

Name : Muhammad Owais
Reg No : FA24-BSE-054
Instructor : Sir Ehaz Mustafa
Lab #5:

#Infix to Postfix using Stack :

```
1  import java.io.IOException;
2  import java.util.Scanner;
3  public class IntoPost {
4      private stack theStack;
5      private String input;
6      private String output = "";
7      public IntoPost(String in){
8          input = in;
9          int stackSize = input.length();
10         theStack = new stack(stackSize);
11     }
12     public String doTrans(){
13         for(int j=0;j<input.length();j++){
14             char ch = input.charAt(j);
15             switch(ch){
16                 case '+':
17                 case '-':
18                     gotOper(ch,prec1: 1);
19                     break;
20                 case '*':
21                 case '/':
22                     gotOper(ch,prec1: 2);
23                     break;
24                 case '(':
25                     theStack.push(ch);
26                     break;
27                 case ')':
28                     gotParen(ch);
29                     break;
30                 default:
31                     output = output + ch;
32                     break;
33             }
34         }
35         while(!theStack.isEmpty()){
36             output = output + theStack.pop();
37         }
38         return output;
39     }
40 }
```

```
40     public void gotOper(char opThis,int prec1){
41         while(!theStack.isEmpty()){
42             char opTop = (char) theStack.pop();
43             if(opTop == '('){
44                 theStack.push(opTop);
45                 break;
46             }
47             else{
48                 int prec2;
49                 if(opTop == '+' || opTop == '-'){
50                     prec2 = 1;
51                 }
52                 else{
53                     prec2 = 2;
54                 }
55                 if(prec2 < prec1){
56                     theStack.push(opTop);
57                     break;
58                 }
59                 else{
60                     output = output + opTop;
61                 }
62             }
63         }
64         theStack.push(opThis);
65     }
66     public void gotParen(char ch){
67         while(!theStack.isEmpty()){
68             char chx = (char) theStack.pop();
69             if(chx == '('){
70                 break;
71             }
72             else{
73                 output = output + chx;
74             }
75         }
76     }
77 }
```

```
78     public static void main(String [] args) throws IOException {
79         String input;
80         Scanner sc = new Scanner(System.in);
81         System.out.print("Enter infix expression: ");
82         input = sc.nextLine();
83         IntoPost theTrans = new IntoPost(input);
84         String output = theTrans.doTrans();
85         System.out.println("Postfix expression: "+output);
86     }
```

```

87     class stack{
88         private int maxSize;
89         private char[] stackArray;
90         private int top;
91         public stack(int size){
92             this.maxSize = size;
93             stackArray = new char[maxSize];
94             top = -1;
95         }
96         public boolean isEmpty(){
97             if(top==-1){
98                 return true;
99             }
100            else{
101                return false;
102            }
103        }
104        public boolean isFull(){
105            if(top==maxSize-1){
106                return true;
107            }
108            else{
109                return false;
110            }
111        }
112        public void push(char value){
113            if(isFull()){
114                System.out.println("Stack is full, cannot push value");
115            }
116            else{
117                top++;
118                stackArray[top]=value;
119                System.out.println("Pushed value: "+value);
120            }
121        }
122        public char pop(){
123            if(isEmpty()){
124                System.out.println("Stack is empty, cannot pop value");
125                return (char) -1;
126            }
127            else{
128                char value = stackArray[top];
129                top--;
130                System.out.println("Popped value: "+value);
131                return value;
132            }
133        }
134        public char peek(){
135            if(isEmpty()){
136                System.out.println("Stack is empty, cannot peek value");
137                return (char) -1;
138            }
139            else{
140                System.out.println("Top element is: "+stackArray[top]);
141                return stackArray[top];
142            }
143        }
144    }

```

#OUTPUT :

```
Enter infix expression: 4*5-(60/6)+5+4*7
Pushed value: *
Popped value: *
Pushed value: -
Pushed value: (
Popped value: (
Pushed value: (
Pushed value: /
Popped value: /
Popped value: (
Popped value: -
Pushed value: +
Popped value: +
Pushed value: +
Popped value: +
Pushed value: +
Pushed value: *
Popped value: *
Popped value: +
Postfix expression: 45*606/-5+47*+
```

#Infix to Prefix :

```
1  import java.util.Scanner;
2  public class intopre {
3      private stack theStack;
4      private String input;
5      private String output = "";
6      public intopre(String in) {
7          input = reverseAndSwap(in);
8          int stackSize = input.length();
9          theStack = new stack(stackSize);
10     }
11     public String reverseAndSwap(String str) {
12         String reversed = "";
13         for (int i = str.length() - 1; i >= 0; i--) {
14             char ch = str.charAt(i);
15             if (ch == '(')
16                 reversed += ')';
17             else if (ch == ')')
18                 reversed += '(';
19             else
20                 reversed += ch;
21         }
22         return reversed;
23     }
24     public String doTrans() {
25         for (int j = 0; j < input.length(); j++) {
26             char ch = input.charAt(j);
27             switch (ch) {
28                 case '+':
29                 case '-':
30                     gotOper(ch, prec1: 1);
31                     break;
32
33                 case '*':
34                 case '/':
35                     gotOper(ch, prec1: 2);
36                     break;
37
38                 case '(':
39                     theStack.push(ch);
40                     break;
41
42                 case ')':
43                     gotParen();
44                     break;
45
46                 default:
47                     output = output + ch;
48                     break;
49             }
50         }
51         while (!theStack.isEmpty()) {
52             output = output + theStack.pop();
53         }
54         return reverse(output);
55     }
56     public void gotOper(char opThis, int prec1) {
57         while (!theStack.isEmpty()) {
58             char opTop = theStack.pop();
59             if (opTop == '(') {
60                 theStack.push(opTop);
61                 break;
62             }
63             else {
64                 int prec2;
65                 if (opTop == '+' || opTop == '-')
66                     prec2 = 1;
67                 else
68                     prec2 = 2;
69                 if (prec2 < prec1) {
70                     theStack.push(opTop);
71                     break;
72                 }
73             }
74             else {
75                 output = output + opTop;
76             }
77         }
78         theStack.push(opThis);
79     }
80     public void gotParen() {
81         char ch = theStack.pop();
82         if (ch != '(')
83             return;
84         while (!theStack.isEmpty())
85             output = output + theStack.pop();
86     }
87 }
```



```

80     public void gotParen() {
81         while (!theStack.isEmpty()) {
82             char chx = theStack.pop();
83             if (chx == '(')
84                 break;
85             else
86                 output = output + chx;
87         }
88     }
89     public String reverse(String str) {
90         String rev = "";
91         for (int i = str.length() - 1; i >= 0; i--) {
92             rev += str.charAt(i);
93         }
94         return rev;
95     }
96
97     Run | Debug | Run main | Debug main
98     public static void main(String[] args) {
99         Scanner sc = new Scanner(System.in);
100         System.out.print("Enter infix expression: ");
101         String input = sc.nextLine();
102         intopre theTrans = new intopre(input);
103         String output = theTrans.doTrans();
104         System.out.println("Prefix expression: " + output);
105     }
106     class stack {
107         private int maxSize;
108         private char[] stackArray;
109         private int top;
110         public stack(int size) {
111             this.maxSize = size;
112             stackArray = new char[maxSize];
113             top = -1;
114         }
115         public boolean isEmpty() {
116             return top == -1;
117         }
118         public boolean isFull() {
119             return top == maxSize - 1;
120         }
121         public void push(char value) {
122             if (!isFull()) {
123                 stackArray[++top] = value;
124             }
125         }
126         public char pop() {
127             if (!isEmpty()) {
128                 return stackArray[top--];
129             }
130             return (char) -1;
131         }
132         public char peek() {
133             if (!isEmpty()) {
134                 return stackArray[top];
135             }
136             return (char) -1;
137         }
138     }

```

#OUTPUT :

```
Enter infix expression: 6*7/5-(5+4)
Prefix expression: -*6/75+54
```